Portfolio Document

1. Full Crow’s Foot ERD: A professional diagram of your 10-table fortress.

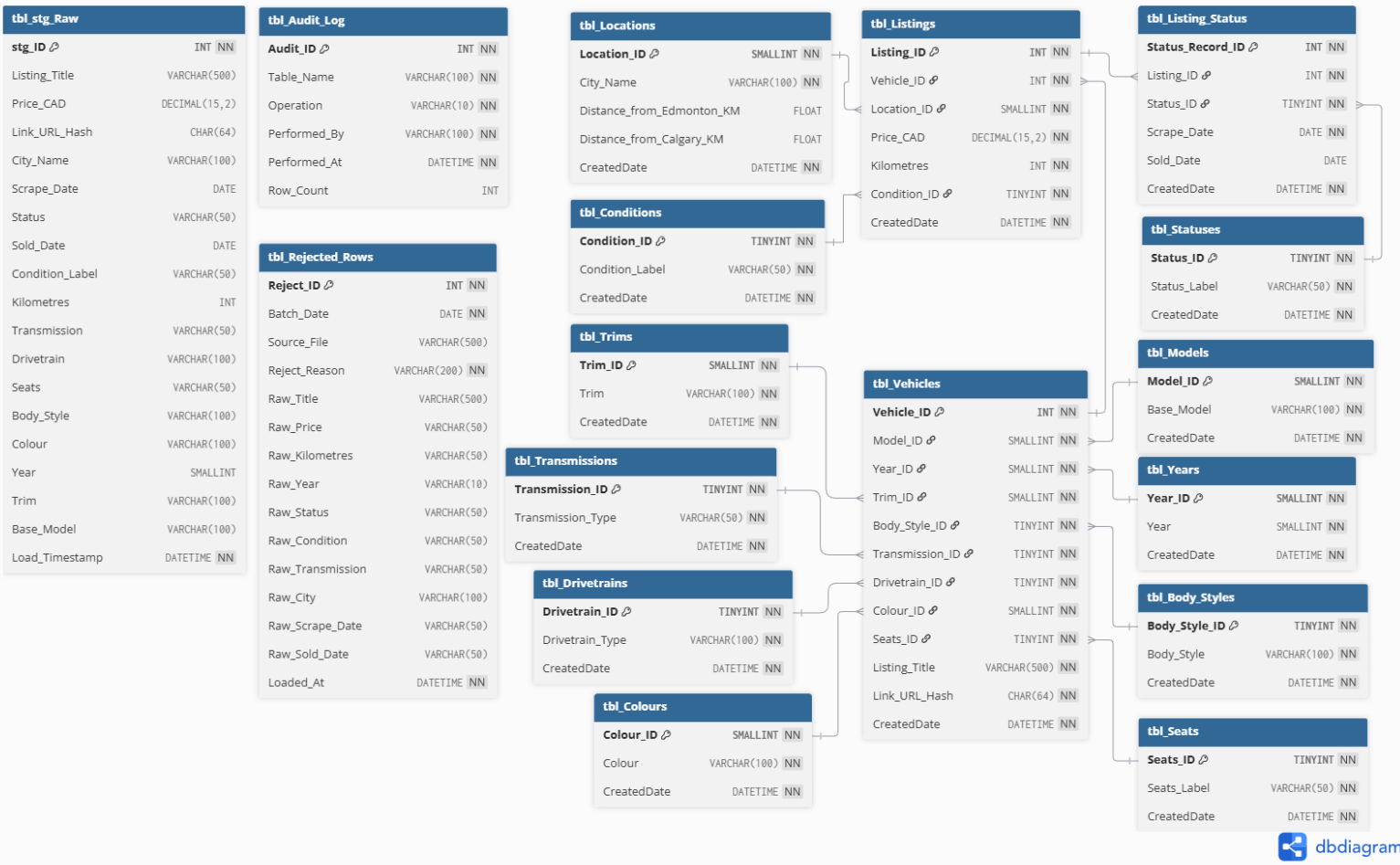

Figure 1: Full Crow’s Foot Entity-Relationship Diagram (ERD) illustrating the 10+ table 3NF SQL Fortress, featuring optimized data types, referential integrity constraints, and ETL audit tables.

Data Dictionary: To complement the visual ERD and satisfy the strict requirements of our Data Fortress, the following data dictionary defines the core business attributes, optimized data types, and constraint rules applied at the database level to ensure data integrity.

Table Name	Column Name	Data Type	Allow NULL	Default Value	Business Meaning	Constraint Rules
tbl_Models	Base_Model	VARCHAR(100)	No	'UNKNOWN'	Vehicle base model name	Length > 0
tbl_Years	Year	SMALLINT	No	None	Manufacturing year	Between 1900 and 2027
tbl_Trims	Trim	VARCHAR(100)	No	'UNKNOWN'	Specific trim level/package	Length > 0
tbl_Locations	City_Name	VARCHAR(100)	No	None	City where vehicle is located	Length > 0
tbl_Locations	Distance_from_Edmonton_KM	FLOAT	Yes	None	Driving distance to Edmonton	>= 0
tbl_Locations	Distance_from_Calgary_KM	FLOAT	Yes	None	Driving distance to Calgary	>= 0
tbl_Statuses	Status_Label	VARCHAR(50)	No	None	Current status of the listing	'SOLD', 'ACTIVE', 'ACTIVE_REPOST', 'RESHELVED'
tbl_Conditions	Condition_Label	VARCHAR(50)	No	'UNKNOWN'	Vehicle condition classification	'USED', 'DAMAGED', 'SALVAGE', 'LEASE TAKEOVER', 'UNKNOWN'
tbl_Transmissions	Transmission_Type	VARCHAR(50)	No	'UNKNOWN'	Type of transmission	'AUTOMATIC', 'MANUAL', 'SEMI-AUTOMATIC', 'OTHER', 'UNKNOWN'
tbl_Drivetrains	Drivetrain_Type	VARCHAR(100)	No	'UNKNOWN'	Drivetrain configuration	Length > 0
tbl_Body_Styles	Body_Style	VARCHAR(100)	No	'UNKNOWN'	Physical body style of vehicle	Length > 0

tbl_Colours	Colour	VARCHAR(100)	No	'UNKNOWN'	Exterior vehicle color	Length > 0
tbl_Seats	Seats_Label	VARCHAR(50)	No	'UNKNOWN'	Number of seats	None
tbl_Vehicles	Listing_Title	VARCHAR(500)	No	None	Original raw title from the ad	Length > 0
tbl_Vehicles	Link_URL_Hash	CHAR(64)	No	None	SHA-256 hashed URL for PII masking	Exact length 64
tbl_Listings	Price_CAD	DECIMAL(15,2)	No	None	Seller's asking price (in CAD)	Between 1 and 10,000,000
tbl_Listings	Kilometres	INT	No	None	Vehicle mileage (in kilometres)	Between 0 and 2,000,000
tbl_Listing_Status	Scrape_Date	DATE	No	None	Date the listing was scraped	>= '2020-01-01'
tbl_Listing_Status	Sold_Date	DATE	Yes	None	Date the listing was sold	>= Scrape_Date
(All Tables)	CreatedDate	DATETIME	No	GETDATE()	ETL insertion timestamp	Auto-populated by SQL Server

2. ETL Architecture Flowchart: Diagram Source - Python - SQL.

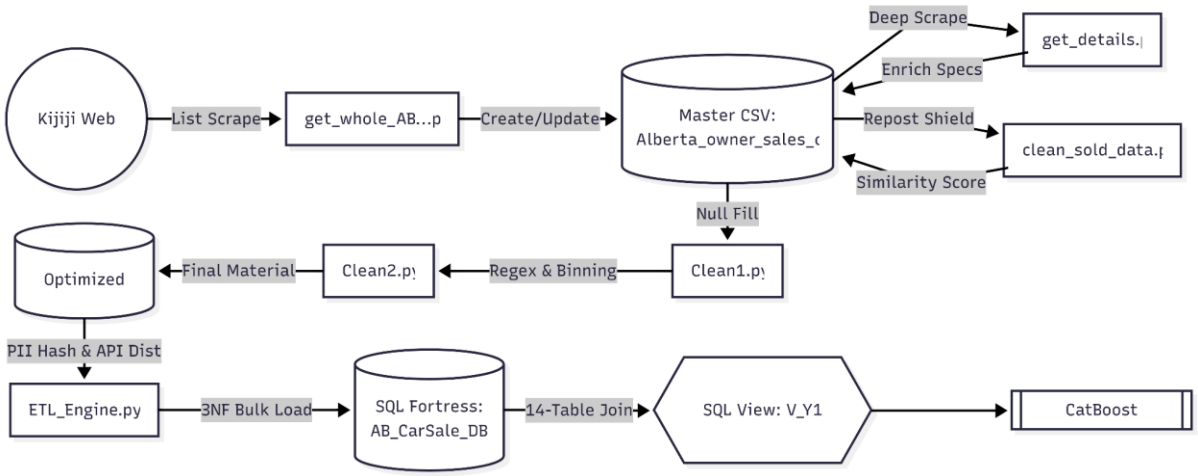

Figure 2: Automated ETL Architecture Flowchart detailing the end-to-end data pipeline, illustrating multi-region web ingestion, heuristic-based repost detection via similarity scoring, regex-driven feature extraction with cardinality optimization, and secure transactional loading featuring SHA-256 PII masking.

Step 1: Orchestrated Web Ingestion (Python)

- The pipeline begins with a multi-region Python scraper (get_whole_...py) targeting 11 Alberta cities on Kijiji.
- It identifies new listings and marks records as "Sold" based on link disappearance between daily scrapes.
- A secondary deep-scraper (get_details.py) then crawls individual ad pages to extract granular technical specs like transmission and drivetrain.

Step 2: Anti-False-Positive Engine (Data Integrity)

- To prevent "Sold" metrics from being inflated by sellers deleting and reposting ads, the clean_sold_data.py script applies a heuristic scoring model.
- By comparing title similarity, price ratios, and odometer readings within a 24-hour window, the system reclassifies phantom "Sold" events as "Active_Repost," ensuring high-fidelity labels for the ML model.

Step 3: Feature Engineering & Cardinality Optimization

- The transformation layer (Clean2.py) decomposes raw string data into structured features, including Year, Trim, and Base_Model.
- Crucially, a binning strategy is applied where categorical variables with fewer than 10 occurrences (e.g., rare trims or remote locations) are grouped into an "OTHER" bucket.
- This significantly reduces feature sparsity and prevents model overfitting during the machine learning phase.

Step 4: Transactional Loading (SQL Fortress)

- The finalized CSV is ingested into a 10-table 3NF schema using an ACID-compliant engine.
- This phase handles complex foreign key mapping, PII masking via SHA-256 hashing for privacy, and automated auditing via SQL Triggers.
- The process results in a model-ready, 14-table joined view (V_Y1) optimized for feature extraction.

3. The "Accuracy Paradox" Defense: Why accuracy was NOT your primary metric.

This is a regression problem — we predict a continuous dollar value (Price_CAD), not a class label. Accuracy is mathematically undefined for regression tasks. Our primary metrics and their business meaning:

Metric	Business Meaning
R^2 (R-Squared)	What percentage of price differences between cars does the model explain? R^2 = 0.89 means 89% of why one car costs more than another is captured by our features.
MAE (Mean Absolute Error)	On average, our prediction is off by $3,039 CAD. This is what a seller actually cares about — how wrong is the system in dollar terms?
RMSE (Root Mean Squared Error)	Penalizes large errors more than small ones. A $20,000 mistake is far worse than two $10,000 mistakes. RMSE captures this asymmetry.

Our final model results demonstrate strong performance across all three metrics:

Metric	CV Result (5-Fold)	Held-Out Test Result
R^2	0.7557 ± 0.0277	0.8886
MAE	$4,161 ± $174	$3,039 CAD
RMSE	$7,711 ± $567	$5,593 CAD

4. Cost-of-Error Analysis: Define the specific cost of a False Negative (FN).

False Negative (Underpricing):

The model predicts $18,000 when the true market value is $22,000. The seller lists too low, the car sells immediately, and the seller permanently loses $4,000 in

potential revenue. This is the most financially damaging error because the money cannot be recovered after the sale.

False Positive (Overpricing):

The model predicts $25,000 when the true market value is $21,000. The car sits unsold for weeks. The seller can reduce the price and relist. The cost is time and inconvenience, not permanent financial loss.

Our three pricing strategies directly address both risks:

- Quick Sale (−10%) — protects against overpricing; guarantees a faster sale
- Balanced — the model's best estimate of fair market value
- Max Profit (+8%) — lets the seller test the upper bound; accepts a longer listing period

5. SQL View Logic Walkthrough: Explain the query used for ML features.

SQL View Logic Walkthrough: Feature Extraction via V_Y1

To bridge the gap between our strict 3NF Data Fortress and the data ingestion requirements of the Machine Learning pipeline, a comprehensive SQL View named V_Y1 was engineered. The primary objective of this view is to flatten the highly normalized schema into a single, wide feature matrix without exposing the underlying transactional tables to raw analytical queries.

Core Query Logic & Massive Table JOINs:

Exceeding the project requirement of a 5-table join, V_Y1 executes a massive 14-table INNER JOIN to reconstruct the complete vehicle profiles. It centralizes the fa

ct tables (tbl_Listings and tbl_Vehicles) and maps their Foreign Keys back to human-readable categorical strings and derived numerical features, successfully delivering exactly 18,614 verified "SOLD" records to the ML pipeline.

Dynamic Filtering Architecture:

Unlike hardcoding a specific status filter inside the view, V_Y1 is designed as a full-dataset view. This allows the ML pipeline to dynamically filter for "SOLD" records during model training, while effortlessly reusing the exact same view to pull "ACTIVE" records for live price inference.

Serving Derived Features (API Integration):

Instead of calculating geographical distances iteratively in Python during model training, the view directly serves the pre-calculated distance metrics from Edmonton and Calgary. This drastically reduces preprocessing latency for the ML team.

Optimized Categorical Retrieval:

CatBoost natively handles categorical variables highly efficiently. The view seamlessly un-normalizes the IDs (such as Model_ID or Colour_ID) back to descriptive strings, allowing the ML engine to immediately apply its internal Target Encoding without requiring complex mapping dictionaries.

6. Hyperparameter Log: Document results of tuning experiments.

The following table documents the final hyperparameter configuration for the Y1 CatBoost price prediction model and the rationale for each choice.

Parameter	Value	Rationale
iterations	2000	Maximum boosting rounds. Early stopping automatically finds the optimal number, so 2000 is a safe upper bound.
learning_rate	0.05	Low learning rate with high iterations — standard CatBoost best practice for stable convergence.
depth	8	Tree depth of 8 captures complex feature interactions (e.g. year × model × condition) without overfitting.
l2_leaf_reg	3.0	L2 regularization term on leaf values; controls overfitting.
subsample	0.8	80% row sampling per tree; reduces variance and speeds up training.
early_stopping_rounds	50	Training stops automatically if no improvement for 50 consecutive rounds. Removes the need for manual iteration tuning.
loss_function	RMSE	Standard regression loss. Penalizes large errors more than MAE, which is appropriate for price prediction.
cat_features	9 columns	Native categorical encoding via CatBoost’s internal Target Encoding. No One-Hot Encoding required.

Parameter	Value	Rationale
random_seed	42	Fixed seed for full reproducibility across all training runs.

Model Comparison Results (5-Fold Cross-Validation)

Model	CV R^2	CV MAE	CV RMSE	Decision
Dummy Baseline	-0.0010	$10,661	$15,633	Rejected — no predictive power
Random Forest	0.6940	$4,442	$8,649	Rejected — lower than CatBoost
CatBoost	0.7557	$4,161	$7,711	Selected as production model

CatBoost was selected because it achieved the highest R^2, lowest MAE, and lowest RMSE across all five folds. It also handles categorical features natively without One-Hot Encoding, and early stopping eliminates the need for manual iteration tuning.

7. Inference Instruction: A guide for processing new rows for prediction.

Prerequisites

- Python 3.9 or higher installed
- SQL Server with AB_CarSale_DB populated via ETL_Engine.py

- ODBC Driver 17 for SQL Server installed
- OpenRouteService API key (free tier available at openrouteservice.org)

Step 1 — Install Dependencies

```
pip install streamlit catboost scikit-learn joblib pandas requests python-dotenv sqlalchemy pyodbc
```

Step 2 — Configure the API Key

Create a file named .env in the project root directory containing:

```
ORS_API_KEY=your_openrouteservice_key_here
```

Step 3 — Verify the Trained Model

Ensure catboost_price_model.pkl exists in the models/ folder. To retrain from scratch, run:

```
python training/Y1_model_catboost.py
```

Step 4 — Launch the Application

```
streamlit run app.py
```

The application opens automatically in the browser. The user enters vehicle details and selects a city to receive a real-time price prediction with three pricing strategy recommendations.

How a New Record is Processed at Inference Time

Step	Action	Component
1	User enters vehicle details and city name in the Streamlit UI	app.py
2	City name geocoded to (longitude, latitude) via open-meteo API	predict.py
3	Road distances to Edmonton and Calgary fetched via OpenRouteService API	predict.py
4	Feature row assembled: vehicle attributes + two distance values	predict.py
5	Saved CatBoost pipeline loaded from catboost_price_model.pkl	predict.py
6	Prediction generated; three pricing strategies calculated (−10%, market, +8%)	app.py
7	Results displayed to user as three pricing strategy cards	app.py

8. Drift Monitoring Plan: How to detect when the model becomes obsolete.

A model trained on 2023–2024 data will degrade as market conditions change (inflation, new vehicle models, shifting demand patterns). The following signals indicate when retraining is needed.

Signal	Threshold	Meaning	Action
Mean residual on recent SOLD listings	> ±$2,000	Model is systematically over- or under-predicting new data	Retrain immediately
R^2 on most recent 30-day SOLD data	< 0.70	Model performance has degraded significantly from baseline	Retrain immediately
New Base_Model values in active listings	Any unseen model	New vehicle models have entered the Alberta market	Flag and retrain
Year distribution shift (KS test)	p < 0.05	Fleet age distribution has shifted significantly	Retrain within 30 days

Retraining Process

Retraining is straightforward because the ETL pipeline handles incremental data loading automatically:

- New SOLD records are appended to AB_CarSale_DB via the regular ETL pipeline run
- Re-run Y1_model_catboost.py against the updated V_Y1 view to produce a new model
- Replace the catboost_price_model.pkl file with the newly trained model

- No changes to the Streamlit application or inference code are required

9. Ethical Sign-off: Reflection on privacy and data protection.

In the development of this vehicle pricing intelligence system, data privacy and the ethical handling of web-scraped information were prioritized. While used car listings are public, they often contain or link to Personal Identifiable Information (PII) such as seller contact details or precise neighborhood locations. To mitigate the risk of exposing seller identities or enabling malicious re-identification, a strict PII masking protocol was implemented directly within the ETL pipeline's transformation phase.

Specifically, the system intercepts the original web URL of the listing and applies a one-way SHA-256 cryptographic hash before it ever reaches the database. As a result, the transactional tbl_Vehicles table stores a 64-character hash rather than the raw URL. This ensures that the Machine Learning model and subsequent analytics dashboards can uniquely identify and track listings without storing any traceable digital footprints back to individual citizens, fully aligning with modern data protection standards.

10. Project Retrospective: Scaling considerations for 1 million+ rows.

While the current ETL architecture successfully ingests, validates, and routes the dataset into a strict 3NF fortress, scaling this pipeline to handle 1,000,000+ rows requires addressing several operational bottlenecks. The current pipeline establishes a strong foundation by utilizing chunked bulk inserts (1000 rows per batch) and SQLAlchemy's fast_executemany parameter to optimize database I/O within

a single transaction block. However, hyper-growth would necessitate three primary architectural upgrades.

First, the API enrichment layer for geographical features represents a significant latency risk. The current pipeline utilizes a synchronous loop with a one-second sleep timer to respect Open-Meteo and ORS API rate limits. Processing one million rows sequentially would take weeks. To resolve this, we would migrate to a locally hosted ORS Docker container for routing and utilize asynchronous Python requests to process coordinates in parallel.

Second, from a database perspective, as tbl_Listings and tbl_Vehicles grow exponentially, the current non-clustered indexes will face fragmentation. We would need to implement Table Partitioning based on CreatedDate or Location_ID to ensure that the massive 14-table JOIN required for the Machine Learning View (V_Y1) remains performant.

Finally, the monolithic single-click ETL script would be decoupled. We would transition the orchestration to a framework like Apache Airflow, breaking the Extract, Transform, API Enrichment, and Load phases into a Directed Acyclic Graph (DAG). This would allow distributed processing and automated failure recovery on a per-task basis, transforming our robust script into an enterprise-grade data engineering pipeline.